SENAI – SERVIÇO NACIONAL DE APRENDIZAGEM INDUSTRIAL

RELATÓRIO DE BANCO DE DADOS

GERENCIAMENTO DE VENDAS

Aluno: Wesley Thibes

Salto Veloso

2025

1. INTRODUÇÃO E CONTEXTUALIZAÇÃO

O projeto consiste na criação de um banco de dados relacional utilizando SQL para organizar as informações de uma empresa de vendas. Ele centraliza os dados de clientes, produtos e pedidos, facilitando o armazenamento, as consultas e as alterações. Também utiliza chaves primárias e estrangeiras para relacionar as tabelas, além de Stored Procedures, Views e Triggers para automatizar tarefas, facilitar consultas e controlar alterações. Dessa forma, o banco de dados torna as informações mais organizadas, seguras e eficientes.

---

2. TIPO DE BANCO DE DADOS E JUSTIFICATIVA

O projeto utiliza um banco de dados relacional com SQL, gerenciado pelo MySQL/MariaDB. As informações são organizadas em tabelas que se relacionam por meio de chaves primárias e estrangeiras. Esse modelo facilita as consultas, reduz a duplicação de dados, aumenta a segurança e permite gerar relatórios e utilizar recursos como Procedures, Views e Triggers.

---

3. TABELAS DO BANCO DE DADOS

As tabelas são uma das principais partes do banco de dados. Elas são responsáveis por armazenar as informações utilizadas pelo sistema.

Neste projeto, foram utilizadas tabelas relacionadas ao funcionamento de uma empresa de vendas.

3.1 Tabela Clientes

A tabela **clientes** tem como objetivo armazenar os dados dos clientes da empresa.

Nela podem ser armazenadas informações como:

- ID do cliente;
- Nome;
- CPF ou CNPJ;
- Rua;
- Número;
- Bairro;
- CEP;
- Referência;
- Telefone;

```sql
CREATE TABLE clientes (
    id_cliente INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    cpf_cnpj VARCHAR(14) NOT NULL,
    rua VARCHAR(255),
    numero VARCHAR(50),
    bairro VARCHAR(100),
    cep VARCHAR(10),
    referencia VARCHAR(255),
    telefone VARCHAR(20)
);
```

Função da tabela:

Armazenar os dados cadastrais dos clientes e identificar cada cliente de forma única.

3.2 Tabela Produtos

A tabela **produtos** é responsável por armazenar as informações dos produtos comercializados pela empresa.

Entre os dados que podem ser armazenados estão:

- ID do produto;
- Nome;
- Preço;
- Estoque;
- Categoria;

```sql
CREATE TABLE produtos (
    id_produto INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    preco DECIMAL(10,2) NOT NULL,
    estoque INT NOT NULL,
    categoria VARCHAR(50)
);
```

Função da tabela:

Armazenar as informações dos produtos comercializados pela empresa.

3.3 Tabela Pedidos

A tabela **pedidos** armazena as informações referentes às compras realizadas.

Ela pode possuir informações como:

- ID do pedido;
- ID do cliente;
- ID do produto;
- Quantidade;
- Número do pedido;
- Data do pedido;

```
CREATE TABLE pedidos (
    id_pedido INT AUTO_INCREMENT PRIMARY KEY,
    id_cliente INT NOT NULL,
    id_produto INT NOT NULL,
    quantidade INT NOT NULL,
    num_ped VARCHAR(255),
    data_ped DATETIME,

    FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente),
    FOREIGN KEY (id_produto) REFERENCES produtos(id_produto)
);*/
```

Nessa tabela são utilizadas **chaves estrangeiras (Foreign Keys)** para criar os relacionamentos com outras tabelas.

Por exemplo:

pedidos → clientes

Um pedido pertence a um determinado cliente.

Também pode existir o relacionamento:

pedidos → produtos

Isso permite identificar qual produto foi comprado.

Função da tabela:

Armazenar os registros das compras realizadas, relacionando clientes e produtos.

---

4. RELACIONAMENTO ENTRE AS TABELAS

O relacionamento entre as tabelas é uma das principais características do banco de dados relacional.

Um exemplo de relacionamento utilizado no projeto é:

CLIENTES → PEDIDOS

Um cliente pode realizar vários pedidos, enquanto cada pedido pertence a um cliente.

Esse relacionamento pode ser representado como:

1 cliente → vários pedidos

Portanto, trata-se de um relacionamento **1:N (um para muitos)**.

---

5. STORED PROCEDURES

As **Stored Procedures**, também chamadas de procedimentos armazenados, são conjuntos de comandos SQL armazenados dentro do próprio banco de dados.

Por exemplo, em vez de escrever várias vezes um comando **`INSERT`** para cadastrar um produto, pode-se criar uma Procedure responsável por realizar esse cadastro.

Uma Procedure pode receber informações através de **parâmetros** e utilizar esses valores para realizar uma determinada operação.

O que cada Procedure faz?

```
BEGIN
INSERT INTO produtos (
    nome,
    preco,
    estoque,
    categoria
  ) VALUES (
    p_nome,
    p_preco,
    p_estoque,
    p_categoria
  );
END
```

A **Procedure** `sp_inserir_produto` é responsável por **cadastrar novos produtos no banco de dados**.

Ela recebe informações como **nome, preço, quantidade em estoque e categoria** e utiliza esses dados para inserir um novo registro na tabela `produtos`.

Em resumo:

> Serve para facilitar e padronizar o cadastro de produtos no banco de dados.

```
BEGIN
 INSERT INTO pedidos (
    id_cliente,
    id_produto,
    quantidade,
    num_ped,
    data_ped
  ) VALUES (
    p_id_cliente,
    p_id_produto,
    p_quantidade,
    p_num_ped,
    p_data_ped,
    NOW()
  );
END
```

A **Procedure** `sp_inserir_pedido` é responsável por **registrar novos pedidos realizados pelos clientes**.

Ela recebe o **ID do cliente, ID do produto, quantidade e preço unitário**. Depois, insere essas informações na tabela `pedidos`.

A data do pedido é preenchida automaticamente utilizando `NOW()`, que registra a **data e hora atual**.

Em resumo:

> Serve para registrar as compras realizadas pelos clientes, relacionando o cliente ao produto comprado.

```
BEGIN
INSERT INTO fornecedores (
    nome,
    cpf_cnpj,
    rua,
    numero,
    bairro,
    cep,
    telefone
) VALUES (
    p_nome,
    p_cpf_cnpj,
    p_rua,
    p_numero,
    p_bairro,
    p_cep,
    p_telefone
);
END
```

A **Procedure** `sp_inserir_fornecedor` é responsável por **cadastrar novos fornecedores** no banco de dados.

Ela recebe informações como **nome, CNPJ, telefone e endereço** e insere esses dados na tabela `fornecedores`.

Em resumo:

> Serve para facilitar o cadastro e a organização dos fornecedores da empresa.

---

6. VIEWS

As **Views**, ou visões, são consultas SQL armazenadas que podem ser utilizadas como se fossem tabelas virtuais.

Elas são muito úteis quando uma consulta possui vários comandos `SELECT`, `JOIN`, `GROUP BY`, `SUM`, `COUNT` ou outras operações.

Neste projeto, as Views são utilizadas principalmente para facilitar a análise das vendas.

View de clientes que mais compraram

```sql
CREATE VIEW view_clientes_que_mais_compraram AS
SELECT
    c.id_cliente,
    c.nome,
    COUNT(p.id_pedido) AS total_compras
FROM clientes c
INNER JOIN pedidos p
    ON c.id_cliente = p.id_cliente
GROUP BY
    c.id_cliente,
    c.nome
ORDER BY total_compras DESC;
```

Essa View pode apresentar informações como:

- Nome do cliente;
- Quantidade de compras realizadas;
- Total de pedidos.

Quais clientes realizaram mais compras?

id_cliente	nome	total_compras
1	João Silva	14
2	Maria Oliveira	14
3	Carlos Pereira	14
4	Ana Santos	14
5	Empresa ABC Ltda	14

View do produto mais vendido

```sql
CREATE VIEW view_produto_mais_vendido AS
SELECT
   pr.id_produto,
   pr.nome,
   SUM(p.quantidade) AS quantidade_vendida
FROM produtos pr
INNER JOIN pedidos p
   ON pr.id_produto = p.id_produto
GROUP BY
   pr.id_produto,
   pr.nome
ORDER BY quantidade vendida DESC;
```

Outra View pode ser utilizada para identificar os produtos que tiveram maior quantidade de vendas em determinado período.

Ela pode apresentar:

- Produto;
- Quantidade vendida;
- Período da venda.

Essa consulta permite identificar:

Qual foi o produto mais vendido no último mês?

id_produto	nome	quantidade_vendida
18	Batedeira Planetária Oster 750W	32
8	Headset Gamer HyperX Cloud II	30
13	Micro-ondas Electrolux 20L	30
1	Smartphone Samsung Galaxy A54	28
20	Ventilador de Mesa Arno Silence Force	28

View de faturamento por cliente

```sql
CREATE VIEW view_faturamento_por_cliente AS
SELECT
   c.id_cliente,
   c.nome,
   SUM(p.quantidade * pr.preco) AS faturamento
FROM clientes c
INNER JOIN pedidos p
   ON c.id_cliente = p.id_cliente
INNER JOIN produtos pr
   ON p.id_produto = pr.id_produto
GROUP BY
   c.id_cliente,
   c.nome
ORDER BY faturamento DESC;
```

Essa View permite calcular o valor total comprado por cada cliente.

Ela pode apresentar:

- Nome do cliente;
- Quantidade de pedidos;
- Valor total das compras.

Qual o valor total de vendas por cliente?

id_cliente	nome	faturamento
7	Mariana Souza	64.317,2
12	Juliana Alves	55.781,4
4	Ana Santos	52.776,2
2	Maria Oliveira	52.204,78
8	Tech Soluções ME	48.372,0

Assim, a empresa consegue identificar quanto cada cliente movimentou em vendas.

---

7. TRIGGERS

Os **Triggers**, também conhecidos como gatilhos, são comandos executados automaticamente pelo banco de dados quando ocorre determinado evento.

Os eventos mais comuns são:

- `INSERT`;
- `UPDATE`;
- `DELETE`.

Isso significa que o banco pode executar uma determinada ação automaticamente quando um registro é inserido, alterado ou excluído.

Trigger de estoque

Um exemplo utilizado no projeto é um Trigger relacionado aos pedidos e ao estoque.

Quando um pedido é inserido, o Trigger pode diminuir automaticamente a quantidade do produto disponível no estoque.

```
BEGIN
   UPDATE produtos
   SET estoque = estoque - NEW.quantidade
   WHERE id_produto = NEW.id_produto;
END
```

Por exemplo:

Estoque = 50

E é realizado um pedido de:

Quantidade = 3

O Trigger pode atualizar automaticamente o estoque para:

Estoque = 47

Dessa forma, não é necessário alterar manualmente o estoque depois de cada venda.

Trigger de Atualização do Estoque nas Compras

A Trigger de atualização do estoque nas compras tem como objetivo atualizar automaticamente a quantidade de produtos disponíveis no estoque quando uma nova compra de fornecedor é registrada na tabela **`compras`.**

Quando um novo registro é inserido na tabela `compras`, a Trigger utiliza o comando **`UPDATE`** para localizar o produto correspondente na tabela `produtos` e acrescentar ao estoque a quantidade comprada.

Funcionamento da Trigger:

1. Uma nova compra é registrada na tabela `compras`.
2. A Trigger é executada automaticamente.
3. O banco de dados identifica o produto por meio do seu ID.
4. A quantidade comprada é adicionada ao estoque atual do produto.

```
BEGIN
    UPDATE produtos
    SET estoque = estoque + NEW.quantidade
    WHERE id_produto = NEW.id_produto;
END
```

Exemplo prático:

Se um produto possui 20 unidades em estoque e uma nova compra registra mais 10 unidades, a Trigger atualiza automaticamente o estoque para 30 unidades.

Importância: essa Trigger automatiza a entrada de mercadorias no estoque, reduz a necessidade de atualizações manuais e contribui para manter as informações do banco de dados atualizadas.

---

8. CONSULTAS E ANÁLISES

Quais clientes realizaram mais compras?

clientes (5r × 3c)

#	id_cliente	cliente	quantidade_compras
1	1	João Pedro	14
2	2	Maria Oliveira	14
3	3	Carlos Pereira	14
4	4	Ana Santos	14
5	5	Empresa ABC Ltda	14

Qual produto foi mais vendido no último mês?

produtos (5r × 3c)

#	id_produto	produto	quantidade_vendida
1	11	Smart TV 55" 4K UHD Samsung	10
2	1	Smartphone Samsung Galaxy A54	8
3	16	Fritadeira Elétrica Air Fryer Mondial 4L	8
4	10	Caixa de Som JBL Flip 6	8
5	3	Notebook Dell Inspiron 15	6

Qual o valor total de vendas por cliente?

clientes (5r × 3c)

#	id_cliente	cliente	valor_total_vendas
1	7	Mariana Souza	5.500,0
2	2	Maria Oliveira	4.190,0
3	12	Juliana Alves	4.080,0
4	13	Comércio de Alimentos LTDA	4.070,0
5	5	Empresa ABC Ltda	4.070,0

Quais pedidos ultrapassam determinado valor?

pedidos (5r × 5c)

#	id_pedido	id_cliente	quantidade	preco_unitario	valor_total
1	**19**	14	5	200,0	1.000,0
2	**104**	5	5	200,0	1.000,0
3	**44**	7	5	200,0	1.000,0
4	**89**	7	5	200,0	1.000,0
5	**34**	13	4	200,0	800,0

Pedidos acima de R$ 500,00.
